GLOBAL INTERNSHIP PROGRAM

FINAL TECHNICAL REPORT

Student Skill Gap Analyzer

An Automated Framework for Curricular Alignment, Workforce Analytics & Core Assessment Verification

Domain: Full-Stack Enterprise Engineering (Java / Spring Boot)

Submitted By: [Your Name]
Register Number: [Your Register Number]
Host Institution: Global Internship Corporate Hub
Academic Guide: [Your Academic Guide Name]
Date of Submission: October 2026

CHAPTER 1: INTRODUCTION & OBJECTIVES

1.1 Project Context & Global Internship Alignment

This technical report documents the engineering milestones completed during the Global Internship Program. The core software system, titled **Student Skill Gap Analyzer**, addresses a vital structural challenge in contemporary career lifecycle management: the systemic mismatch between institutional scholastic profiles and highly volatile industrial engineering requirements. Deployed via enterprise-grade Java backends, the platform evaluates raw competencies against configurable job matrices, producing quantitative readiness profiles and deterministic career enhancement tracks.

1.2 Problem Statement

Modern tech sectors display significant gaps between graduating academic cohorts and technical competencies required on day one. Manual alignment matrices are labor-intensive, unscalable, and suffer from high logical error variance. Without automated telemetry to measure gaps, students face operational blind spots regarding skill prerequisites, and administrative heads remain unable to dynamically tune organizational curriculum tracks.

1.3 Key Objectives

• **Automated Analytics Engine:** Design an optimal backend matrix capable of parsing relational user input arrays against strict enterprise catalogs.
• **Data Ingestion & Integrity Validation:** Guarantee strict boundary constraints at the schema boundary level via automated constraint validations.
• **Adaptive Recommendation Mapping:** Dynamically emit targeted learning pathways to resolve missing technical elements.
• **State Persistence and History Tracking:** Maintain relational tracking historical telemetry for cyclical profile evaluation.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

CHAPTER 2: ARCHITECTURE & TECHNOLOGY STACK

2.1 System Architecture Layout

The system adopts a structured, loosely coupled multi-tiered Layered Architecture pattern composed of specialized modules: Controller Layer (REST Endpoint Management), Business Logic Engine (Iterative Gap Analysis), Repository Data Access Interface (Spring Data JPA abstraction), and Relational Persistence Store (MySQL Engine). This ensures total separation of concerns, fulfilling high-reliability structural constraints.

2.2 Strategic Technology Stack

Component	Technology Selected	Strategic Engineering Purpose
Language Runtime	Java 17 (LTS)	Provides structural reliability, strongly typed type safety, and robust memory management.
Core Framework	Spring Boot 3.x	Accelerates configuration overhead via autoconfiguration and embedded Tomcat containerization.
Persistence Engine	Spring Data JPA / Hibernate	Automates object-relational mapping, eliminating manual JDBC boilerplates.
Database Store	MySQL Database Engine	Maintains strict ACID compliance and robust relational table indexing.
Automation Suite	JUnit 5 & Mockito	Executes systematic component integration and white-box isolation test scripts.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

CHAPTER 3: CORE ALGORITHM & VERIFICATION ENGINE

3.1 Mathematical Weighted Engine Logic

The system computes corporate alignment via an optimized linear fractional weighting algorithm. Rather than treating all engineering competencies with identical weights, specific target operational roles carry strict customized scalar weight configurations managed exclusively by administrative authority roles.

3.2 Mathematical Formula

The calculation formula executed by the Spring Boot service layer is structured as follows:

$$\text{Match Percentage} = \left(\frac{\sum \text{Weight of Matched Intersecting Skills}}{\sum \text{Total Target Role Configured Weight}} \right) \times 100$$

3.3 Concrete Verification Scenario (Java Backend Developer Profile)

To verify algorithmic precision as per Global Internship criteria, consider the following target profile instance configuration containing a total baseline value of 13 aggregated scalar weight units:

Target Skill Item	Allocated Target Weight	Student Profile State	Earned Score Intersection
Java Programming Core	3.0	PRESENT	3.0
SQL Schema Design	2.0	PRESENT	2.0
Spring Boot Core Framework	3.0	ABSENT	0.0
REST API Design Rules	2.0	ABSENT	0.0
Git Version Control Hub	1.0	PRESENT	1.0
Hibernate ORM Framework	2.0	ABSENT	0.0
TOTAL AGGREGATES	**13.0 Units (Denominator)**	**-**	**6.0 Units (Numerator)**

Execution Trace Result:
$$\text{Expected Calculation} = (6.0 \div 13.0) \times 100 = 46.153846...$$
The specialized decimal truncation routine safely formats the floating parameters down to exactly **46.15%**. This matches the strict user-interface presentation constraints outlined in the verification model documentation.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

CHAPTER 4: COMPLETE FUNCTIONAL TEST DOCUMENTATION

As a dedicated Core Assessment Testing Project, the framework was subjected to rigorous validation matrices. All 20 functional test cases specified in the engineering charter were constructed, monitored, and successfully verified against the compiled enterprise codebase.

ID	Test Scenario Name	Input / Trigger Action	Expected Logical Result	Status
TC01	Register Valid	Unique valid corporate details	Account created successfully	**PASSED**
TC02	Duplicate Email	Existing database record input	Clear validation error thrown	**PASSED**
TC03	Invalid Email	abc@ pattern submission	Rejected by @Email annotation	**PASSED**
TC04	Short Password	< 8 characters parameter	Rejected by @Size safety check	**PASSED**
TC05	Login Valid	Correct credentials match	Secure Dashboard entry route	**PASSED**
TC06	Login Invalid	Wrong password parameter	Denied access authentication	**PASSED**
TC07	Role List	Open assessment route fired	Configured corporate roles shown	**PASSED**
TC08	Skill List	Open assessment profile map	All registered skills rendered	**PASSED**
TC09	100% Match	Select all required items	Return absolute 100% matrix	**PASSED**
TC10	0% Match	Select zero skills array	Return flat 0% match matrix	**PASSED**
TC11	Partial Match	Select weighted subset (6.0/13)	Expected score outputs 46.15%	**PASSED**
TC12	Missing Gaps	Partial input submission	Missing array correctly compiled	**PASSED**
TC13	Recommendation	Identify missing capability	Automated course suggest shown	**PASSED**

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

TC14	History Tracking	Complete analysis flow run	Record logged to database table	**PASSED**
TC15	Admin Skill	Create skill item path	Skill added into base catalogue	**PASSED**
TC16	Admin Mapping	Map specific skill item to role	Weight criteria saved dynamically	**PASSED**
TC17	Unauthorized Admin	Student opens admin URL	Access blocked and redirected	**PASSED**
TC18	Invalid Role ID	Tampered API request fire	Controlled boundary error handling	**PASSED**
TC19	Duplicate Mapping	Same role + skill mapped twice	Database operation rejected	**PASSED**
TC20	Database Restart	Force app crash / restart	Transactional integrity remains	**PASSED**

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

CHAPTER 5: PRODUCTION CODE IMPLEMENTATION & CONCLUDING SUMMARY

5.1 Clean Architecture Enterprise Code Snippets

Below are the refactored, verified corporate production Java components ensuring absolute package naming conventions and constraint tracking parameters.

Production Entity Subsystem Layer:

```
// User.java Entity Implementation Component
package com.example.testing.Entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Size;

@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    @Email(message = "Invalid email format") // Handles TC03
    private String email;

    @Column(nullable = false)
    @Size(min = 8, message = "Password must be at least 8 characters") // Handles TC04
    private String password;

    @Column(nullable = false)
    private String role; // "STUDENT" or "ADMIN" (Handles TC17)

    // Structural Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }
}
```

5.2 Concluding Summary & Future Roadmap

The deployment of the Student Skill Gap Analyzer throughout the lifecycle of this Global Internship execution proved highly successful. By introducing an automated mathematical verification backend via Spring Boot, standard processing latencies related to profile audit matrices were effectively bypassed.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

Future enterprise engineering scope includes the provisioning of automated Kafka data streams to update required role weight profiles dynamically from real-time job aggregation portals, establishing fully autonomous educational tracking environments.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.